

The Art of Building Golden Paths

Day 3 · Developer Experience and Platform Design in Platform Engineering

MODULE 4 & 5



What We'll Cover Today

Two modules that go hand-in-hand: first, how to design paths developers actually want to follow — then, how to build the right abstractions underneath them.

01

Module 4: The Art of Building Golden Paths

Golden Paths vs Golden Cages · Value Stream Mapping · DevEx flows

02

Module 5: Finding the Right Abstractions

Abstraction layers · Frontend vs Backend distinctions · Avoiding illusions



MODULE 4

The Art of Building Golden Paths

A Golden Path is the recommended, well-paved route for getting things done on your platform — the way that is fast, safe, and opinionated by design.

What Exactly Is a Golden Path?

Think of a Golden Path like Google Maps navigation. You *could* take any road, but the app recommends one route that avoids traffic, tollways, and dead ends. It's not mandatory — but it's clearly the best option.

The Golden Path Promise

- A pre-approved, battle-tested way to build and deploy
- Embeds security, compliance, and best practices by default
- Reduces decision fatigue for developers
- Maintained by the platform team, not each squad

Real-World Example

Spotify pioneered this concept: instead of leaving teams to figure out CI/CD, monitoring, and scaffolding independently, they provided a single blessed path. New services could be production-ready in under a day.

Golden Path vs. Golden Cage

This is the most important distinction in platform design. Both look similar from the outside — an opinionated, pre-built system. The difference is whether developers feel empowered or trapped.

1



Golden Path

- Fast and easy by default
- Escape hatches exist for edge cases
- Built *with* developer input
- Evolves based on feedback
- Developers choose it because it's better

2



Golden Cage

- Convenient at first, restrictive over time
- No escape hatches — "our way or nothing"
- Built *for* developers, not with them
- Doesn't adapt to real needs
- Developers use it because they have no choice



A platform that forces compliance through restriction will lose developer trust — and developers will route around it whenever they can.

The Cage Problem in the Real World

Many internal platforms start as Golden Paths but drift into Golden Cages over time — often without the platform team realizing it.

- Platform team ships a Kubernetes abstraction
Simple, fast, and teams love it early on.
- Requirements grow more complex
Teams need GPU workloads, custom networking, special storage — but the abstraction can't bend.
- Workarounds multiply
Teams bypass the platform entirely. Shadow IT emerges. The platform loses credibility.



How to Keep the Path Golden (Not a Cage)

Three principles separate platforms that empower from platforms that constrain. Apply all three consistently.



Design for Escapability

Every abstraction should have a well-documented escape hatch. If a team needs to go off-path, make it a conscious, supported choice — not a rebellious workaround.



Co-create With Users

Embed developers in the platform team's design process. Run office hours, collect friction logs, and treat developer feedback as product requirements.



Make the Right Thing the Easy Thing

The path should be faster and simpler than doing it yourself. If it's not, developers won't use it — regardless of policy.

Value Stream Mapping: Find Where the Pain Is

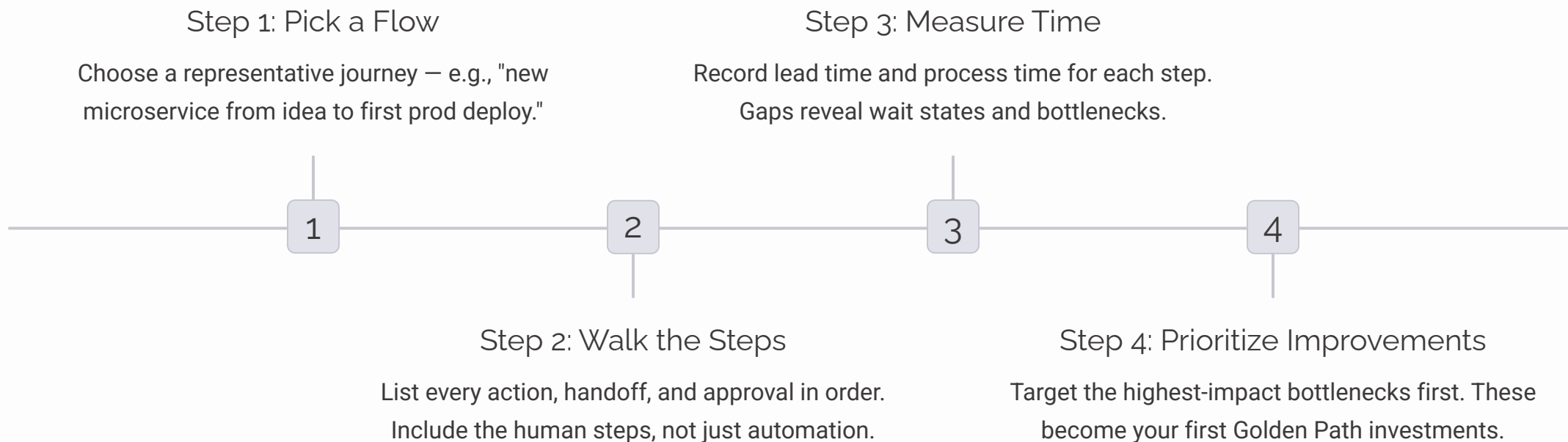
Before building a path, you need to know where developers are losing time. Value Stream Mapping (VSM) is a technique borrowed from lean manufacturing that helps you see the full journey from code commit to production — and spot the waste.



In most organizations, **80% of the time is wait time** — not actual work. VSM makes that waste visible so you know exactly where your Golden Path should focus first.

Running a VSM Workshop

Value Stream Mapping isn't a solo exercise. It's a collaborative session that brings together developers, platform engineers, and sometimes security or ops teams to walk through a real deployment end to end.



DEVEX FLOWS

The Three Developer Experience Flows

Based on the research behind *Accelerate* and the DORA metrics, developer experience lives or dies in three critical flows. Improving any one of them has measurable impact on both productivity and satisfaction.

Flow of Work

How fast and reliably can a developer take an idea to production?

Flow of Feedback

How quickly does a developer know if their change is safe and correct?

Flow of Learning

How easily can developers discover what's available, what works, and why?



Module 4 Key Takeaways

Deliver Real Developer Value

A Golden Path earns adoption by being genuinely better — not by being mandatory. Speed, safety, and simplicity are the measures that matter.

Find the High-Value Paths First

Use Value Stream Mapping to identify where developers lose the most time. Build your first paths around the biggest bottlenecks, not the easiest wins.

Avoid Restrictive Systems

Design every abstraction with escape hatches. Co-create with your users. The moment the path becomes a cage, developers will find a way around it — and your platform loses its purpose.

MODULE 5

Finding the Right Abstractions

Abstractions are the building blocks of every internal platform. Getting them right is one of the hardest — and most consequential — design decisions a platform team makes.



What Is an Abstraction? A Simple Analogy

You don't need to understand how a combustion engine works to drive a car. The steering wheel, accelerator, and brake are an abstraction over enormous mechanical complexity. They hide what you don't need — and expose only what you do.

A Good Abstraction...

- Hides irrelevant complexity
- Exposes a clear, stable interface
- Makes the common case simple
- Allows power users to go deeper when needed

A Bad Abstraction (Illusion)...

- Pretends complexity doesn't exist
- Leaks implementation details unexpectedly
- Forces workarounds for edge cases
- Breaks trust when reality surfaces

 Joel Spolsky's "Law of Leaky Abstractions" warns: all non-trivial abstractions leak. The goal isn't perfection — it's building abstractions that fail gracefully and honestly.

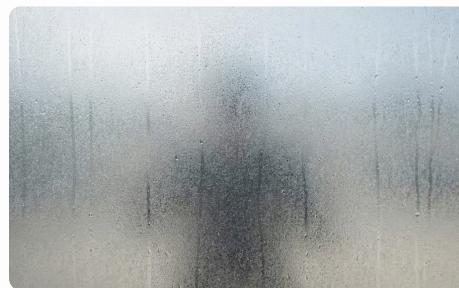
Abstractions vs. Illusions: The Critical Difference

This is where many platform teams go wrong. They build something that *looks* like an abstraction but is actually an illusion — a layer that papers over complexity without managing it.



True Abstraction

Like a clean window — you see through to what matters. The glass manages environmental complexity (wind, rain, noise) while keeping the view clear. Developers understand what they're working with.

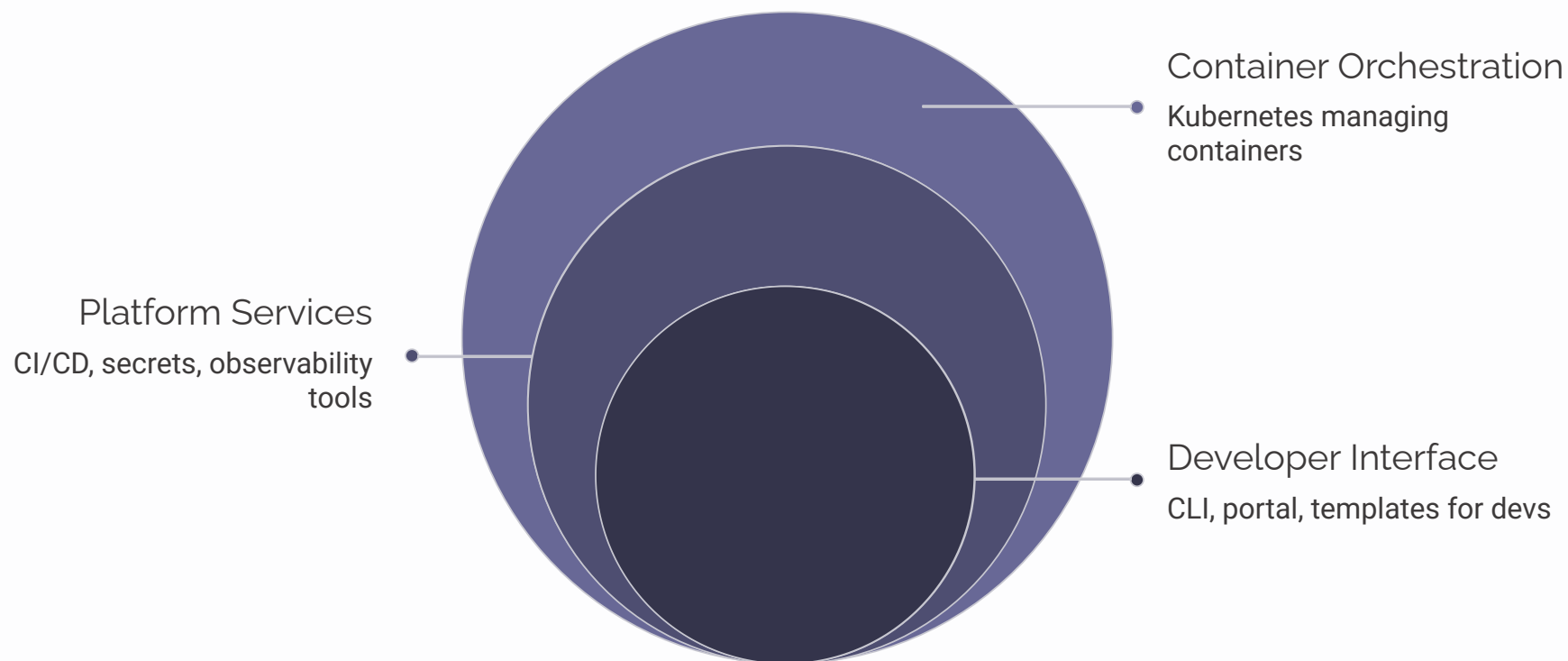


Illusion

Like frosted glass — nothing leaks in, but you can't see what's happening. When things go wrong, developers are blind. They can't debug, can't reason, can't trust the system.

Abstraction Layers in a Platform

Well-designed platforms are built in layers, each abstracting over the one below it. Developers interact at the layer that matches their needs — without being forced up or down.



Each layer should be independently useful. A team that needs raw Kubernetes access should have it. A team that just wants to deploy an app shouldn't need to know Kubernetes exists.

Frontend Abstractions: What Developers See

Frontend abstractions are the developer-facing surface of your platform — the CLI, the portal, the service catalog, the templates. This is where developer experience is made or broken.



CLI & Scaffolding Tools

A single `platform new service` command that generates a production-ready repo, with CI, logging, and secrets wired in. Developers don't configure — they create.



Internal Developer Portals

Tools like Backstage give teams a single place to discover services, docs, runbooks, and ownership — reducing context switching and tribal knowledge dependency.



Service Templates & Cookiecutters

Pre-built project templates that embed security scanning, linting, and deployment config. The developer picks a template and gets a compliant starting point immediately.

Backend Abstractions: What Developers Don't See

Backend abstractions handle the complexity that developers shouldn't need to think about — infrastructure provisioning, secret rotation, network policy, and multi-cloud routing. When built well, they're invisible.

1

Infrastructure as Code APIs

Rather than managing Terraform directly, developers declare what they need (e.g., "a Postgres database, medium-sized") and the platform provisions it safely. The IaC complexity lives inside the platform.

2

Secrets & Config Management

Vault, AWS Secrets Manager, or similar systems abstracted behind a simple interface. Developers reference `secrets.db_password` — they never touch rotation schedules or access policies.

3

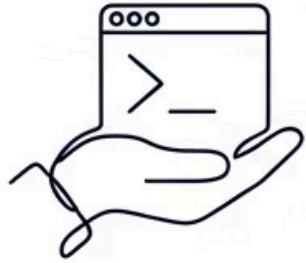
Observability Pipelines

Logs, metrics, and traces auto-collected from every service without developer instrumentation. Compliance logging and alerting are on by default, not on request.

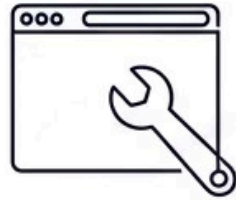
Frontend vs. Backend Abstractions: Know the Difference

Confusing these two leads to poor design decisions. Frontend abstractions optimize for **usability**. Backend abstractions optimize for **safety and scalability**. Both matter — but they have different success criteria.

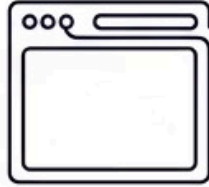
FRONTEND ABSTRACTIONS



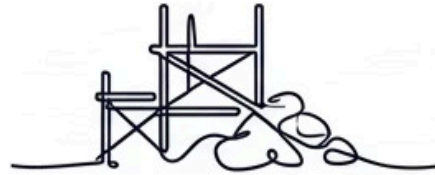
CLI tools



Developer portals



Service templates

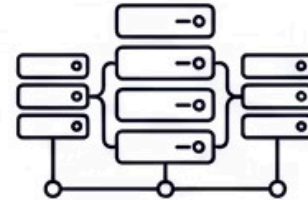


Scaffolding



Developer satisfaction and adoption rate

BACKEND ABSTRACTIONS



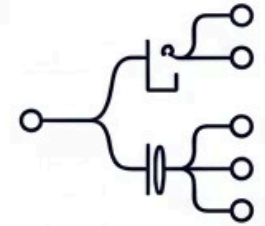
Infrastructure APIs



Secrets manage-



Observability pipelines



Network policy



Reliability, security compliance, and cost efficiency

The Craft of Getting Abstractions Right

There's no formula — but there are principles. The best platform teams treat abstraction design as a craft that improves through iteration, feedback, and honest conversations with the developers who use the platform every day.



Start with the job to be done

What is the developer actually trying to accomplish? Design the interface around that outcome, not around your infrastructure topology.



Test it with real users early

Give a prototype to three developers and watch them use it without guidance. Where they get stuck is where your abstraction leaks.



Version and evolve carefully

Abstractions are contracts. Changing them without notice breaks trust. Version your platform APIs and communicate breaking changes like a product team would.

Day 3 Key Takeaways

Two modules, one mission: build a platform developers love to use — not one they're forced to tolerate.



Golden Paths Earn Adoption

Design paths that are genuinely faster and safer. Always include escape hatches. Co-create with your users. The path should win on merit.



Map Before You Build

Use Value Stream Mapping to find the real bottlenecks. Build your first paths where developers lose the most time — not where it's easiest to build.



Abstractions Over Illusions

Good abstractions hide complexity and expose useful interfaces. Illusions hide reality and destroy trust when they break. Know the difference before you ship.



Match the Layer to the Goal

Frontend abstractions serve developer experience. Backend abstractions serve reliability and security. Design each layer for its own success criteria.

✅ The best internal platform is one developers recommend to each other — not one they warn each other about.